

Efficient $e = 3$ Threshold RSA via Integer Coordinates for Intel SGX

Sam Ng¹ and Jason Lau²

¹ samngms@gmail.com

² jason@zeroclicks.ai

Abstract. Threshold RSA signatures face a fundamental obstacle: reconstructing the private exponent from Shamir shares requires Lagrange coefficients whose computation involves modular division by values tied to $\phi(N)$, which must remain hidden. This obstacle is particularly acute for critical deployments such as Intel SGX code signing, which mandates $e = 3$. Existing $e = 3$ -compatible approaches incur substantial overhead, increased share sizes, or sacrifice security properties such as perfect secrecy. This work introduces the integer coordinate framework, achieving $e = 3$ support with EUF-CMA security under the standard RSA assumption alone. By carefully selecting interpolation coordinates that yield integer-valued Lagrange coefficients, we eliminate all modular inversions modulo $\phi(N)$, requiring only standard integer arithmetic and modular exponentiation. The framework achieves $O(\kappa)$ -bit share sizes, perfect secrecy, and computational efficiency previously unattained for $e = 3$ -compatible schemes.

Although we currently lack an efficient general algorithm for constructing coordinate sets for arbitrary (t, n) —a challenging open problem for future work—the coordinate families found via heuristic search achieve coverage for $2 \leq t \leq n \leq 9$ and selected $n = 10$ configurations, sufficient for small boardroom-size deployments. The resulting online protocol is extremely simple and immediately enables practical $e = 3$ threshold RSA for Intel SGX and similar applications.

1 Introduction

1.1 Motivation and Background

Threshold cryptography [8, 5] enables cryptographic operations to be performed collaboratively by multiple parties without requiring any single entity to possess complete secret key material. This distribution of trust is fundamental to securing critical infrastructure where single points of failure are unacceptable. Threshold RSA signatures [17] are particularly significant given RSA’s pervasive deployment in code signing systems, certificate authorities, and hardware security modules.

Intel SGX enforces a hard requirement that enclave signing keys use public exponent $e = 3$ and will reject signatures using the industry-standard exponent

$e = 65537$ [4]. While threshold RSA schemes such as Shoup [20] support arbitrary exponents efficiently, they cannot be applied to SGX code signing due to this fixed-exponent constraint. This paper’s integer coordinate framework directly addresses the SGX-mandated $e = 3$ requirement, enabling secure threshold signing infrastructure for enclave code.

The application of Shamir Secret Sharing [19] to RSA private exponents encounters a fundamental mathematical obstacle. Standard Lagrange interpolation requires computing coefficients of the form $\lambda_i = \prod_{j \in S, j \neq i} \frac{x_j}{x_j - x_i} \pmod{\phi(N)}$, where S denotes the set of participating parties and $\phi(N)$ is the Euler totient function³ of the RSA modulus N . Security of RSA fundamentally depends on $\phi(N)$ remaining computationally infeasible to compute, creating an inherent tension: threshold reconstruction appears to require arithmetic operations using a value whose secrecy is essential.

1.2 Integer Coordinate Approach: Mathematical Foundation

This work establishes a fundamentally different mathematical framework based on the following observation: for appropriately chosen interpolation coordinates $\{x_1, \dots, x_n\} \subset \mathbb{Z}$, the Lagrange coefficients $\lambda_i(S) = \prod_{j \in S, j \neq i} \frac{x_j}{x_j - x_i}$ evaluated over $\mathbb{Z}$ (without any modular reduction) are integers for all relevant subsets S . This integrality property eliminates all modular arithmetic modulo $\phi(N)$ from the online signing protocol, yielding a reconstruction procedure requiring only integer operations and standard modular exponentiation modulo N .

Theoretical Significance. The existence of such integer coordinate families constitutes a non-trivial mathematical property. For a (t, n) threshold scheme, integrality must hold for all $\binom{n}{t}$ possible signing coalitions simultaneously, imposing a complex system of divisibility constraints on the coordinate selection. Understanding when such families exist, characterizing their minimal growth rates, and developing efficient construction algorithms represent fundamental questions at the intersection of combinatorics, number theory, and cryptographic protocol design.

1.3 Contributions

This work makes the following contributions to the theory and practice of threshold RSA:

1. **Most Efficient EUF-CMA Secure Threshold RSA for $e = 3$:** Among schemes supporting $e = 3$, this work provides the first scheme to achieve EUF-CMA security (unforgeability) against malicious adversaries under the standard RSA assumption alone, with standard $O(\kappa)$ -bit share sizes and

³ More precisely, Carmichael’s totient function $\lambda(N)$ [18] suffices. For expository clarity, $\phi(N)$ is employed throughout this work; the mathematical framework applies identically to $\lambda(N)$.

perfect secrecy. The framework achieves universal parameter compatibility for discovered coordinate families—supporting $e = 3$ for all threshold configurations with $n \leq 9$ without restrictions on t (unlike prior approaches requiring $n > 2t$)—without requiring additional cryptographic primitives. Comprehensive comparison (Table 1) demonstrates substantial advantages over existing $e = 3$ -compatible approaches.

2. **Integer Coordinate Framework with Proven Unforgeability:** Formal establishment of the integer coordinate paradigm for threshold RSA, including complete correctness proofs and EUF-CMA security analysis against static malicious adversaries under the standard RSA assumption (Theorem 1). The framework provides a clean separation between unforgeability (proven) and robustness (practical extension discussed in Appendix A).
3. **Complete Coordinate Characterization for $n \leq 9$ and Structural Phenomena:** Construction of integer coordinate families providing universal coverage for (t, n) configurations with $2 \leq t \leq n \leq 9$, along with selected high-threshold configurations for $n = 10$ (Section 4). This demonstrates immediate applicability to small-to-moderate group sizes common in practical threshold deployments. Empirical identification and characterization of mathematical patterns governing coordinate existence, including lifting properties, multiplicative scaling, and smoothness requirements.
4. **Practical Validation and Comparative Analysis:** Detailed evaluation of existing $e = 3$ -compatible threshold RSA schemes across multiple dimensions: share size, computational cost, security model, perfect secrecy, and hardware suitability. The integer coordinate approach proves ideal for hardware-constrained environments (HSMs, secure enclaves) through elimination of polynomial arithmetic and complex interactive protocols. Implementation and performance analysis demonstrating practical feasibility, including successful deployment on resource-constrained Ledger hardware wallets (Section 6).

1.4 Scope and Limitations

This work addresses the threshold signature reconstruction problem under the following model:

- **Trusted Dealer Model:** A trusted dealer generates RSA keys and distributes shares via secure channels
- **Adversary Model:** Static, malicious (active) adversaries corrupting up to $t - 1$ parties. The core security analysis (Theorem 1) establishes EUF-CMA security (unforgeability) under the standard RSA assumption alone. This is the primary contribution and stands as a complete result.
- **Robustness (Optional Extension):** An optional DLEQ-based verification mechanism is presented in Appendix A for practical deployments requiring robustness against denial-of-service attacks. Formal security analysis of this extension to adaptive adversaries requires careful treatment of witness dependencies and is left as future work.

- **Coordinate Families:** Coordinate families are provided for (t, n) configurations with $n \leq 9$, along with selected configurations for $n = 10$. *We currently do not have an efficient general algorithm for constructing integer coordinate sets for arbitrary threshold sizes beyond $n = 10$.* All coordinate families reported in this work were discovered through ultimately heuristic computational search with some minor optimizations (Section 4). Development of efficient constructive algorithms for larger thresholds represents an important open problem.
- **Out of Scope:** Distributed Key Generation (DKG) [16,11] and proactive refresh [13]

The integer coordinate framework’s public coordinate structure naturally supports optional extensions for verification and robustness. The DLEQ-based approach (Appendix A) has been successfully deployed in practice and provides pragmatic security, though formal proofs under all attack models remain open. Further extensions such as DKG and proactive refresh are compatible with the framework and constitute future work.

1.5 Organization

The remainder of this paper is organized as follows. Section 2 analyzes prior threshold RSA approaches and their limitations. Section 3 establishes notation, formal definitions, and the security model. Section 4 presents the main construction, including formal algorithms, correctness proofs, and the coordinate search methodology. Section 5 provides comprehensive security analysis establishing the unforgeability result. Section 6 reports implementation results and performance characteristics. Section 7 concludes with discussion of open problems and future directions. Appendix A presents an optional DLEQ-based robustness extension for deployments requiring verification of partial signatures, including detailed algorithms (Algorithm 3–6) and security discussion.

2 Related Work

Threshold RSA signature research has developed along three primary directions: denominator elimination techniques, polynomial ring-based share representation, and parameter restriction approaches. This section analyzes these approaches and their fundamental limitations, particularly regarding support for small public exponents.

2.1 Denominator Elimination Approaches

Shoup’s Fundamental Construction. The foundational work of Shoup [20] established the denominator elimination paradigm through multiplicative clearing. The scheme computes modified Lagrange coefficients $\Delta \cdot \lambda_i$ where $\Delta = n!$, ensuring integrality through the divisibility property: for any subset S of size t

and any $i \in S$, the denominator $\prod_{j \in S \setminus \{i\}} (x_j - x_i)$ divides Δ when standard coordinates $x_i = i$ are employed.

The critical constraint is $\gcd(e, 4\Delta^2) = 1$. For $e = 3$, this requires $3 \nmid 4\Delta^2$, equivalently $3 \nmid \Delta = n!$. Since $3 \mid n!$ for all $n \geq 3$, the scheme excludes $e = 3$ with $n > 2$ participants. This algebraic incompatibility renders Shoup’s approach unsuitable for applications mandating $e = 3$ [20,14].

Refined Denominator Analysis. Subsequent work has refined the denominator elimination technique. Damgård and Dupont [6] observe that the least common multiple of all possible denominators is strictly smaller than $n!$ for many coordinate choices, permitting a reduced clearing factor Δ' . However, the fundamental constraint $\gcd(e, 4(\Delta')^2) = 1$ persists, maintaining restrictions on small exponents: when e is small relative to n , the coprimality condition remains difficult to satisfy.

Damgård and Koprowski [7] extend denominator elimination to generalized secret sharing structures and develop techniques for trustless setup via distributed key generation. Their protocol maintains the core algebraic framework and inherits similar exponent restrictions. The polynomial structure analysis required to ensure simultaneous denominator elimination and RSA parameter validity introduces additional complexity without fundamentally resolving the small-exponent limitation.

2.2 Polynomial Ring-Based Share Representation

Cyclotomic Polynomial Framework. Frankel and Desmedt [10] introduce a fundamentally different algebraic structure by embedding shares in polynomial rings $\mathbb{Z}_{\phi(N)}[X]/(\Phi_r(X))$, where $\Phi_r(X)$ denotes the r -th cyclotomic polynomial [21]. Share values are polynomials of degree $O(\max(t, n))$ rather than field elements. The cyclotomic structure permits precomputation of all required inversions during share generation, eliminating online modular arithmetic modulo $\phi(N)$.

This approach successfully avoids exponent restrictions: $e = 3$ is fully compatible with the framework. However, the representation incurs substantial overhead. Share size grows linearly with the polynomial degree, and reconstruction requires polynomial arithmetic in the quotient ring rather than simple field operations. For a (t, n) threshold scheme, each share comprises $\Theta(n)$ field elements, and combining operations require $O(n^2)$ multiplications in $\mathbb{Z}_{\phi(N)}$. This complexity increase renders the approach less attractive for resource-constrained deployments despite its theoretical elegance.

2.3 Collaborative Inversion Framework: Catalano et al.

Catalano et al. [2] introduce a fundamentally distinct approach that circumvents the modular inversion problem through collaborative computation over a publicly shared auxiliary modulus, entirely independent of $\phi(N)$. The protocol has participating parties jointly compute modular inverses through multiple rounds of interaction, enabling compatibility with arbitrary public exponents including

$e = 3$ without the algebraic constraints of denominator elimination. However, the approach imposes the structural restriction $n > 2t$ on threshold configurations, excluding high-threshold scenarios such as $(n - 1)$ -of- n or n -of- n deployments. The joint inversion protocol requires $O(n^2)$ message complexity across multiple interaction rounds, increasing both latency and implementation complexity compared to non-interactive approaches.

2.4 Extended-Modulus Approaches

Xu and Chen [22] develop an alternative extended-modulus approach that achieves $e = 3$ compatibility by performing Shamir secret sharing over an auxiliary modulus r (chosen such that $e \nmid r$ and $\gcd(r, \phi(N)) = 1$) rather than over $\mathbb{Z}_{\phi(N)}$. The scheme computes shares as $d_i = f(i) \bmod r$ where $f(X)$ is a degree- $(t - 1)$ polynomial with $f(0) = d \bmod r$. During reconstruction, parties compute partial results $y_i = m^{d_i} \bmod N$, and an additional exponentiation with precomputed auxiliary parameter k yields the final signature $m^d \bmod N$. This approach supports arbitrary thresholds including $(n - 1)$ -of- n configurations.

Potential Security Concern: The Xu-Chen scheme may have a potential information leakage concern related to the perfect secrecy property of Shamir secret sharing. In standard Shamir secret sharing over $\mathbb{Z}_{\phi(N)}$, any $t - 1$ shares reveal zero information about the secret d or about any other party's share. However, working over $\mathbb{Z}_r$ instead of the natural domain $\mathbb{Z}_{\phi(N)}$ may create additional algebraic structure that could potentially be exploited:

- Consider a $(3, 3)$ -threshold configuration with evaluation points $x_1 = 1, x_2 = 2, x_3 = 3$, where party 3 is the sole honest participant while parties 1 and 2 are corrupted.
- During secret reconstruction, the Lagrange coefficient for party 3 is $\lambda_3 = \frac{2}{2-3} \cdot \frac{1}{1-3} = 1$, implying that $y_3 = d - (\lambda_1 y_1 + \lambda_2 y_2) \pmod{r}$.
- The corrupted coalition can compute $\lambda_1 y_1 + \lambda_2 y_2 \pmod{r}$. If, for example, the value equals approximately $\frac{3r}{2}$ before reduction modulo r , yielding a value near $\frac{r}{2}$ after reduction, then observing certain values in the reconstruction protocol could potentially reveal whether a modular wrap-around occurred.
- Under specific parameter choices and coalition structures, this observation could potentially leak information about the most significant bit of the honest party's share y_3 .

While a rigorous analysis with formal probability bounds and concrete simulations would be needed to establish the practical severity of this concern, the structural difference of working over $\mathbb{Z}_r$ rather than $\mathbb{Z}_{\phi(N)}$ warrants careful consideration. The scheme maintains unforgeability (malicious parties cannot forge signatures), but whether it fully preserves the information-theoretic privacy guarantees of threshold cryptography—where $t - 1$ parties should learn nothing about honest parties' shares beyond what valid signatures reveal—requires further investigation.

Table 1. Comprehensive comparison of threshold RSA schemes. Notation: κ = security parameter (RSA modulus bit-length), n = total parties, t = threshold, $\Delta = n!$.

Scheme	$e = 3$ Support	Share Size (bits)	Online Ops (per party)	Secrecy
Shoup [20]	No ($e \nmid 4\Delta^2$)	$O(\kappa)$	1 exp.	Yes
Damgård-Dupont [6]	No ($e \nmid 4(\Delta')^2$)	$O(\kappa)$	1 exp.	Yes
Frankel-Desmedt [10]	Yes	$O(n\kappa)$ (polynomial)	$O(n^2)$ poly. ops	Yes
Catalano et al. [2]	Yes ($n > 2t$)	$O(\kappa)$	$O(n^2)$ msgs. interactive	Yes
Xu-Chen [22]	Yes	$O(\kappa)$	1 exp. + 2 comb. exp.	Concern (potential)
Joye-Michalevsky [14]	Yes (conversion)	N/A (single-party)	N/A (non-threshold)	N/A
Integer Coordinates (This work)	Yes (universal)	$O(\kappa)$ (standard)	1 exp.	Yes

2.5 Application-Specific Workarounds

Joye and Michalevsky [14] address the specific challenge of RSA signatures with $e = 3$ in Hardware Security Modules (HSMs) through signature conversion. The technique generates signatures under an exponent compatible with existing threshold schemes (typically $e = 65537$) and applies an algebraic transformation yielding signatures verifiable under $e = 3$. This approach resolves the immediate compatibility issue for single-signer HSM deployments but does not extend to threshold scenarios, as the conversion itself is not distributable among multiple parties. The method underscores the practical significance of $e = 3$ support in deployed cryptographic systems.

2.6 Comparative Analysis

Table 1 summarizes the key characteristics of the threshold RSA approaches discussed in the preceding subsections.

3 Preliminaries

3.1 Notation

The following notation is employed throughout:

- $\kappa \in \mathbb{N}$ denotes the security parameter
- $N = pq$ where p, q are distinct primes of bit-length $\Theta(\kappa)$; N is the RSA modulus

- $\phi(N) = (p-1)(q-1)$ denotes the Euler totient function
- $e \in \mathbb{Z}^+$ denotes the public exponent satisfying $\gcd(e, \phi(N)) = 1$
- $d \in \mathbb{Z}_{\phi(N)}$ denotes the private exponent satisfying $ed \equiv 1 \pmod{\phi(N)}$
- $t, n \in \mathbb{N}$ with $2 \leq t \leq n$ denote threshold and total party count respectively
- $[n] := \{1, 2, \dots, n\}$ denotes the party index set
- $\mathcal{P} = \{P_1, \dots, P_n\}$ denotes the party set
- $S \subseteq [n]$ with $|S| = t$ denotes a signing coalition
- $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{Z}$ denotes the coordinate family with $x_i \neq x_j$ for $i \neq j$
- $f(X) = \sum_{j=0}^{t-1} a_j X^j \in \mathbb{Z}_{\phi(N)}[X]$ denotes the secret sharing polynomial with $a_0 = d$
- $y_i = f(x_i) \bmod \phi(N) \in \mathbb{Z}_{\phi(N)}$ denotes the share of party P_i
- $\lambda_{i,S} = \prod_{j \in S \setminus \{i\}} \frac{x_j}{x_j - x_i}$ denotes the Lagrange coefficient for party i under coalition S ; subscript S is omitted when clear from context
- $m \in \mathbb{Z}_N$ denotes a message representative (after encoding via PSS [15] or similar)
- $\sigma \in \mathbb{Z}_N$ denotes a complete RSA signature
- $\sigma_i \in \mathbb{Z}_N$ denotes a partial signature from party P_i
- $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ denotes the binomial coefficient

3.2 Standard Definitions

Definition 1 (RSA Problem). Let $\text{RSAGen}(1^\kappa)$ denote an algorithm that outputs (N, e, d) where $N = pq$ is a product of two κ -bit primes, e is chosen with $\gcd(e, \phi(N)) = 1$, and $d = e^{-1} \bmod \phi(N)$. The RSA problem is: given (N, e) and $y = x^e \bmod N$ for uniformly random $x \in \mathbb{Z}_N^*$, compute x . The RSA assumption states that for all probabilistic polynomial-time (PPT) algorithms $\mathcal{A}$,

$$\Pr \left[(N, e, d) \leftarrow \text{RSAGen}(1^\kappa), x \xleftarrow{\$} \mathbb{Z}_N^*, y \leftarrow x^e \bmod N : \mathcal{A}(N, e, y) = x \right] = \text{negl}(\kappa).$$

Definition 2 (Threshold RSA Signature Scheme). A (t, n) -threshold RSA signature scheme is a tuple of PPT algorithms $(\text{KeyGen}, \text{Share}, \text{PartialSign}, \text{Combine})$:

- $\text{KeyGen}(1^\kappa, e) \rightarrow (N, e, d)$: generates RSA parameters with specified public exponent e
- $\text{Share}(d, \phi(N), t, n, \mathcal{X}) \rightarrow (y_1, \dots, y_n)$: creates shares using coordinate family $\mathcal{X} = \{x_1, \dots, x_n\}$
- $\text{PartialSign}(i, y_i, m, S, \mathcal{X}) \rightarrow \sigma_i$: party P_i generates partial signature for coalition $S \ni i$
- $\text{Combine}(\{\sigma_i\}_{i \in S}, m, N) \rightarrow \sigma$: reconstructs complete signature from partial signatures

Correctness requires that for all $(N, e, d) \leftarrow \text{KeyGen}(1^\kappa, e)$, all $(y_1, \dots, y_n) \leftarrow \text{Share}(d, \phi(N), t, n, \mathcal{X})$, all $S \subseteq [n]$ with $|S| = t$, and all $m \in \mathbb{Z}_N^*$,

$$\text{Combine}(\{\text{PartialSign}(i, y_i, m, S, \mathcal{X})\}_{i \in S}, m, N) = m^d \bmod N.$$

Definition 3 (Shamir Secret Sharing and Lagrange Interpolation). For a degree- $(t - 1)$ polynomial $f(X) = \sum_{j=0}^{t-1} a_j X^j$ over a ring R , given evaluations $\{f(x_i)\}_{i \in S}$ at distinct points $\{x_i\}_{i \in S}$ where $|S| = t$, the constant term is recovered via Lagrange interpolation:

$$f(0) = \sum_{i \in S} f(x_i) \cdot \lambda_{i,S}, \quad \text{where} \quad \lambda_{i,S} = \prod_{j \in S \setminus \{i\}} \frac{x_j}{x_j - x_i}.$$

In threshold RSA, f is defined over $\mathbb{Z}_{\phi(N)}$ with $f(0) = d$. Standard application requires computing $\lambda_{i,S} \bmod \phi(N)$, necessitating knowledge of $\phi(N)$.

Definition 4 (Integer Coordinate Family). A coordinate family $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{Z}$ with distinct elements is (t, n) -integer if for all subsets $S \subseteq [n]$ with $|S| = t$ and all $i \in S$, the Lagrange coefficient

$$\lambda_{i,S} = \prod_{j \in S \setminus \{i\}} \frac{x_j}{x_j - x_i}$$

is an integer when evaluated over $\mathbb{Z}$ (i.e., $\lambda_{i,S} \in \mathbb{Z}$).

The significance of Definition 4 is that when $\mathcal{X}$ is (t, n) -integer, signature reconstruction requires only integer arithmetic and modular exponentiation modulo N , with no dependence on $\phi(N)$. This constitutes the foundational insight of the integer coordinate framework.

Example 1 ((2, 3)-Integer Coordinate Family). Consider the coordinate family $\mathcal{X} = \{3, 4, 6\}$. For each possible 2-element signing coalition $S \subseteq \{1, 2, 3\}$, the Lagrange coefficients are:

$$\begin{aligned} S = \{1, 2\}: \quad \lambda_1 &= \frac{4}{4-3} = 4, \quad \lambda_2 = \frac{3}{3-4} = -3 \\ S = \{2, 3\}: \quad \lambda_2 &= \frac{6}{6-4} = 3, \quad \lambda_3 = \frac{4}{4-6} = -2 \\ S = \{1, 3\}: \quad \lambda_1 &= \frac{6}{6-3} = 2, \quad \lambda_3 = \frac{3}{3-6} = -1 \end{aligned}$$

All coefficients are integers, verifying that $\mathcal{X}$ is $(2, 3)$ -integer. Note that coordinate families are closed under negation: $\{-3, -4, -6\}$ is also $(2, 3)$ -integer, and more generally under uniform scaling by any nonzero integer.

4 Integer Coordinate Threshold RSA Construction

This section presents the complete threshold RSA signature scheme based on integer coordinate families, establishing the core unforgeability result (Theorem 1). The construction assumes a trusted dealer model for key generation and share distribution. An optional robustness extension via DLEQ-based verification is presented separately in Appendix A.

Protocol Description. The scheme is compatible with standard RSA message encoding (PSS [15], PKCS#1, etc.) and requires only that encoded messages m satisfy $\gcd(m, N) = 1$, which holds with overwhelming probability for properly encoded messages.

The integer coordinate threshold RSA signature scheme consists of two primary algorithms presented below. Algorithm 1 describes the key generation and share distribution procedure executed by the trusted dealer, which generates RSA keys, creates Shamir secret shares using the integer coordinate family $\mathcal{X}$, and securely distributes shares to parties. Algorithm 2 specifies the distributed signing protocol, where a coalition of t parties collaboratively generates an RSA signature by computing partial signatures using their shares and the integer Lagrange coefficients, then combining these partial signatures to reconstruct the complete signature—all without any party requiring knowledge of $\phi(N)$.

Algorithm 1 KeyGen($1^\kappa, e, t, n, \mathcal{X}$) – Key Generation and Share Distribution

- 1: **Input:** Security parameter κ , public exponent e , threshold parameters (t, n) , (t, n) -integer coordinate family $\mathcal{X} = \{x_1, \dots, x_n\}$
 - 2: **RSA Key Generation:**
 - 3: Generate random κ -bit primes p, q
 - 4: Compute $N \leftarrow p \cdot q$ and $\phi(N) \leftarrow (p-1)(q-1)$
 - 5: Verify $\gcd(e, \phi(N)) = 1$; if not, regenerate primes
 - 6: Compute $d \leftarrow e^{-1} \bmod \phi(N)$
 - 7: **Secret Sharing:**
 - 8: Sample $a_1, a_2, \dots, a_{t-1} \xleftarrow{\$} \mathbb{Z}_{\phi(N)}$ uniformly at random
 - 9: Define polynomial $f(X) \leftarrow d + \sum_{j=1}^{t-1} a_j X^j \in \mathbb{Z}_{\phi(N)}[X]$
 - 10: For $i = 1, \dots, n$: compute $y_i \leftarrow f(x_i) \bmod \phi(N)$
 - 11: **Distribution:**
 - 12: **for** $i = 1$ to n **do**
 - 13: Securely transmit (x_i, y_i) to party P_i via authenticated private channel
 - 14: **end for**
 - 15: Publish public parameters $(N, e, \mathcal{X})$
 - 16: **Secure Erasure:** Destroy $p, q, \phi(N), d, \{a_j\}_{j=1}^{t-1}$
 - 17: **Output:** Public key $(N, e, \mathcal{X})$; private shares $\{(x_i, y_i)\}_{i=1}^n$ distributed to parties
-

Following the key generation and share distribution in Algorithm 1, the distributed signing protocol (Algorithm 2) enables any coalition S of t parties to collaboratively generate RSA signatures. The key feature of this protocol is that all Lagrange coefficients λ_i are computed as integers over $\mathbb{Z}$ (by virtue of the integer coordinate property of $\mathcal{X}$), eliminating the need for any party to know or compute with $\phi(N)$. Each party computes their partial signature through a single modular exponentiation, and the combiner reconstructs the complete signature by multiplying these partial signatures modulo N .

Key Observations. Algorithm 2 requires no knowledge of $\phi(N)$ for any party. All Lagrange coefficients λ_i are computed as integers over $\mathbb{Z}$ by Definition 4, and

Algorithm 2 ThresholdSign($m, S, \{(x_i, y_i)\}_{i \in S}, N, e, \mathcal{X}$) – Distributed Signature Generation with Malicious Security

- 1: **Input:** Message representative $m \in \mathbb{Z}_N^*$, signing coalition $S \subseteq [n]$ with $|S| = t$, shares $\{(x_i, y_i)\}_{i \in S}$, public key $(N, e, \mathcal{X})$
- 2: **Partial Signature Generation (executed by each P_i for $i \in S$):**
- 3: Compute integer Lagrange coefficient:

$$\lambda_i \leftarrow \prod_{j \in S \setminus \{i\}} \frac{x_j}{x_j - x_i} \in \mathbb{Z}$$

- 4: Compute exponent: $w_i \leftarrow \lambda_i \cdot y_i \in \mathbb{Z}$
 - 5: **if** $\lambda_i > 0$ **then**
 - 6: $\sigma_i \leftarrow m^{w_i} \bmod N$
 - 7: **else** ▷ Handle negative coefficient
 - 8: Compute $m^{-1} \bmod N$ via extended Euclidean algorithm
 - 9: $\sigma_i \leftarrow (m^{-1})^{-w_i} \bmod N$
 - 10: **end if**
 - 11: Broadcast σ_i to all parties in S
 - 12:
 - 13: **Signature Combination:**
 - 14: Compute complete signature: $\sigma \leftarrow \prod_{i \in S} \sigma_i \bmod N$
 - 15: **Output:** RSA signature $\sigma \in \mathbb{Z}_N$
-

exponentiation is performed modulo N only. Negative coefficients (which may arise depending on coordinate ordering) are handled via modular inversion of the message, which is efficiently computable since $\gcd(m, N) = 1$ holds unless m contains a factor of N which is with negligible probability.

Handling Negative Coefficients. When $\lambda_i < 0$, the partial signature computation uses the identity $m^{\lambda_i \cdot y_i} \equiv (m^{-1})^{-\lambda_i \cdot y_i} \pmod{N}$ as shown in Algorithm 2. The modular inverse $m^{-1} \bmod N$ is efficiently computable via the extended Euclidean algorithm. For properly encoded messages (RSA-PSS [15], PKCS#1), $\gcd(m, N) = 1$ holds except with negligible probability $O(2^{-\kappa})$.

Signature Reconstruction Correctness. The complete signature is reconstructed as

$$\sigma \equiv \prod_{i \in S} \sigma_i \equiv \prod_{i \in S} m^{\lambda_i \cdot y_i} \equiv m^{\sum_{i \in S} \lambda_i \cdot y_i} \pmod{N}.$$

By Lagrange interpolation over $\mathbb{Z}_{\phi(N)}$, $\sum_{i \in S} \lambda_i \cdot y_i \equiv d \pmod{\phi(N)}$, ensuring $\sigma \equiv m^d \pmod{N}$ by Euler's theorem.

4.1 Integer Coordinate Construction: The Search Problem

Problem Formalization

Problem 1 (Problem 1 (Coordinate Construction)). Given threshold parameters (t, n) with $2 \leq t \leq n$, determine:

1. **(Existence)** Do there exist (t, n) -integer coordinate families?
2. **(Construction)** If so, provide an efficient algorithm that outputs such a family $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{Z}$.
3. **(Bounds)** Characterize the minimal achievable coordinate magnitudes: what is $\min_{\mathcal{X}} \max_{i \in [n]} |x_i|$ over all (t, n) -integer families?

Divisibility Characterization. A family $\mathcal{X} = \{x_1, \dots, x_n\}$ is (t, n) -integer if and only if for all $S \subseteq [n]$ with $|S| = t$ and all $i \in S$,

$$\prod_{j \in S \setminus \{i\}} (x_j - x_i) \mid \prod_{j \in S \setminus \{i\}} x_j, \quad (1)$$

where $a \mid b$ denotes " a divides b ". This formulation as a divisibility constraint is algorithmically convenient for verification but provides limited insight into constructive existence.

Computational Search Methodology At present, no efficient general algorithm for Problem 1 is known. The coordinate families presented in this work (Table 2) were obtained via heuristic computational search employing the following strategies:

Hierarchical Construction.

1. **Base Case ($t = 2$):** A pair $\{a, b\}$ is $(2, 2)$ -integer if and only if $(b - a) \mid a$ and $(a - b) \mid b$, equivalently $\frac{b}{b-a}, \frac{a}{a-b} \in \mathbb{Z}$. Enumerate small pairs satisfying this condition.
2. **Incremental Extension:** Extend the solution sets by ensuring every pair within a candidate set satisfies the $(t+1, n)$ -integer condition. For example, if $\{a, b\}$, $\{b, c\}$, and $\{a, c\}$ are each valid $(2, 2)$ -integer pairs, then $\{a, b, c\}$ forms a valid $(2, 3)$ -integer set because every 2-subset among $\{a, b, c\}$ satisfies the integrality condition.
3. **Direct Search for $t \geq 3$:** For higher thresholds, direct enumeration begins with (t, t) -integer seeds (complete t -element families) and extends incrementally. This avoids dependence on $(2, n)$ families, as lifting (see below) from $t = 2$ to $t \geq 3$ does not hold universally.

Lifting Phenomenon. Empirically, certain (t, n) -integer families satisfy the (t', n) -integer property for $t' > t$. For instance, the $(2, 9)$ -integer family presented in Table 2 is verified to be $(t, 9)$ -integer for all $t \in \{2, \dots, 9\}$. This phenomenon suggests structural properties warranting theoretical investigation, but no general characterization of liftable families is currently known.

Multiplicative Scaling and Smoothness. Coordinates with highly composite (smooth) structures exhibit higher success rates in satisfying divisibility constraints (1). The $(2, 9)$ -integer family was constructed by scaling a $(2, 8)$ -integer family's largest coordinate by

$$D = 2^{10} \cdot 3^5 \cdot 5^5 \cdot 7 \cdot 11^4 \cdot 13 \cdot 17 \cdot 19 \cdot 29 \cdot 31 \cdot 37 \cdot 41 \cdot 43,$$

yielding the base coordinate $552720 \cdot D = 10846504854867738232560384000000$ in Table 2. This multiplicative augmentation technique—incorporating abundant prime factors to facilitate divisibility—proved effective but lacks theoretical justification or optimality guarantees. The specific exponents (e.g., 2^{10} rather than 2^{20}) were determined empirically through heuristic search and do not possess any special theoretical significance. We observe that successful coordinate families typically incorporate many small prime factors, though the precise values employed here represent heuristic choices rather than theoretically optimal parameters. Following initial discovery, a reduction step is applied to eliminate common factors among the coordinates.

Complexity Analysis and Current Limitations The computational cost of searching for (t, n) -integer families with coordinates bounded by B exhibits the following time complexity:

Exhaustive search over candidate coordinate families scales as $O(B^n)$, where each candidate requires verification of $\binom{n}{t}$ divisibility constraints (1). Each constraint verification involves computing products of up to t terms, requiring $O(t^2 \ln B)$ arithmetic operations. This yields an overall complexity of

$$O\left(B^n \cdot \binom{n}{t} \cdot t^2 \ln B\right).$$

This exponential scaling in the coordinate bound renders brute-force search infeasible for large B or large n . The coordinate families in Table 2 were obtained through heuristic search strategies that exploit divisibility structure and smooth number properties, as described in the preceding subsection.

Existence and Computational Status. The current state of Problem 1 is:

- **Established Results:** Universal (t, n) -integer families for all (t, n) with $2 \leq t \leq n \leq 9$, plus selected high-threshold configurations for $n = 10$ (Table 2).
- **Computational Barrier:** Current heuristic methods are effective for $n \leq 9$ but do not scale to $n \gtrsim 12$ without theoretical breakthroughs. Coordinate magnitude growth (Table 2) suggests potential inherent complexity.

Table 2 summarizes established coordinate families, providing universal coverage for $n \leq 9$ and exemplary high-threshold configurations for $n = 10$.

5 Security Analysis

This section establishes the EUF-CMA security (existential unforgeability under chosen-message attacks) of the integer coordinate threshold RSA scheme under the standard RSA assumption. The analysis considers a static, malicious (active) adversary corrupting up to $t-1$ parties. We prove that the adversary cannot forge signatures, even with full control over corrupted parties’ behavior. Note that the

Table 2. Discovered coordinate sets for various threshold configurations

$(t, 9)$ with $2 \leq t \leq 9$	10846504854867738232560384000000, 10846504854867738232560383773200, 10846504854867738232560383568800, 10846504854867738232560383566245, 10846504854867738232560383552448, 10846504854867738232560383543250, 10846504854867738232560383536680, 10846504854867738232560383510400, 10846504854867738232560383497260
$(t, 8)$ with $2 \leq t \leq 8$	552720, 552825, 552960, 553000, 553014, 553140, 553280, 554400
$(6, 8)$	210, 215, 216, 217, 220, 222, 224, 225
$(7, 9)$	210, 216, 219, 220, 224, 225, 231, 234, 240
$(8, 10)$	210, 216, 220, 224, 225, 228, 230, 231, 234, 240
$(9, 10)$	39, 40, 42, 45, 46, 48, 49, 52, 54, 55
$(10, 10)$	1, 2, 3, 4, 5, 6, 7, 8, 9, 10

proof establishes unforgeability but not robustness: malicious parties may cause signing operations to fail (denial-of-service). Robustness can be achieved through optional verification mechanisms such as the DLEQ-based extension presented in Appendix A.

5.1 Security Model

Definition 5 (Security Model Parameters). *The threshold signature scheme operates under the following trust and adversary model:*

- **Setup:** A trusted dealer executes **KeyGen** and distributes shares via authenticated private channels. After distribution, the dealer is assumed to securely erase all secret material.
- **Adversary Type:** Static, malicious (active) adversary $\mathcal{A}$.
 - Static corruption: $\mathcal{A}$ selects the corruption set $\mathcal{C} \subseteq [n]$ with $|\mathcal{C}| \leq t - 1$ before protocol execution begins
 - Malicious behavior: Corrupted parties may deviate arbitrarily from the protocol specification, including:
 - * Submitting invalid or incorrectly computed partial signatures
 - * Refusing to participate in signing rounds
 - * Attempting to learn information about honest parties' shares through malformed messages
 - * Coordinating arbitrary attacks across corrupted parties
- **Communication Model:** After initial share distribution, all signing operations occur over public authenticated channels. Adversary $\mathcal{A}$ observes all protocol messages and controls corrupted parties' behavior.
- **Security Goal:** Existential unforgeability under chosen-message attacks (EUF-CMA) [12]: $\mathcal{A}$ cannot produce a valid signature on a message not previously signed, even with adaptive access to the signing oracle and control over up to $t - 1$ corrupted parties. Note that malicious parties may cause signing operations to fail by submitting invalid partial signatures (denial-of-service), but this does not enable forgery.

5.2 EUF-CMA Security Game

Definition 6 (EUF-CMA Game for Threshold Signatures). For security parameter κ , threshold parameters (t, n) , and adversary $\mathcal{A}$, the EUF-CMA game $\text{Game}_{\mathcal{A}}^{\text{EUF-CMA}}(\kappa, t, n)$ proceeds as follows:

1. **Setup:** The challenger runs $(N, e, d) \leftarrow \text{KeyGen}(1^\kappa, e)$ and $(y_1, \dots, y_n) \leftarrow \text{Share}(d, \phi(N), t, n, \mathcal{X})$ for a (t, n) -integer family $\mathcal{X}$. The public key $(N, e, \mathcal{X})$ is given to $\mathcal{A}$.
2. **Static Corruption:** Adversary $\mathcal{A}$ selects corruption set $\mathcal{C} \subseteq [n]$ with $|\mathcal{C}| \leq t - 1$. The challenger provides $\{(x_i, y_i)\}_{i \in \mathcal{C}}$ to $\mathcal{A}$.
3. **Signing Queries:** Adversary $\mathcal{A}$ makes adaptive signature queries. For query $m_j \in \mathbb{Z}_N^*$:
 - The challenger selects signing coalition $S_j \subseteq [n]$ with $|S_j| = t$
 - For each $i \in S_j$, the challenger computes partial signature $\sigma_{i,j} \leftarrow \text{PartialSign}(i, y_i, m_j, S_j, \mathcal{X})$
 - The challenger provides $\{\sigma_{i,j}\}_{i \in S_j}$ to $\mathcal{A}$ and computes $\sigma_j \leftarrow \text{Combine}(\{\sigma_{i,j}\}_{i \in S_j}, m_j, N)$
 Let $\mathcal{Q} = \{m_1, m_2, \dots, m_q\}$ denote the set of queried messages.
4. **Forgery:** Adversary $\mathcal{A}$ outputs (m^*, σ^*) .

Adversary $\mathcal{A}$ wins the game if:

- $(\sigma^*)^e \equiv m^* \pmod{N}$ (validity)
- $m^* \notin \mathcal{Q}$ (non-trivial forgery)

The advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}(\kappa, t, n) := \Pr[\mathcal{A} \text{ wins } \text{Game}_{\mathcal{A}}^{\text{EUF-CMA}}(\kappa, t, n)].$$

The threshold signature scheme is EUF-CMA secure if for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}(\kappa, t, n) = \text{negl}(\kappa)$.

5.3 Main Security Theorem

Theorem 1 (EUF-CMA Security Against Malicious Adversaries). Let $\mathcal{X}$ be a (t, n) -integer coordinate family. Under the RSA assumption, the threshold signature scheme (Algorithms 1, 2) is EUF-CMA secure (Definition 6) against static, malicious adversaries corrupting up to $t - 1$ parties.

More precisely, for any PPT adversary $\mathcal{A}$ attacking the threshold signature scheme with arbitrary malicious behavior, there exists a PPT algorithm $\mathcal{B}$ solving the RSA problem such that

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}(\kappa, t, n) \leq \text{Adv}_{\mathcal{B}}^{\text{RSA}}(\kappa) + \text{negl}(\kappa),$$

where the running time of $\mathcal{B}$ is polynomial in that of $\mathcal{A}$ and κ .

Scope of Security Result. This theorem establishes unforgeability: adversaries cannot produce valid signatures on messages they haven't queried, even with control over $t - 1$ parties. The result requires only the standard RSA assumption with no additional cryptographic primitives.

High-Level Strategy. The simulator $\mathcal{S}$ receives an RSA challenge (N, e, y) where $y = x^e \bmod N$ for unknown $x \in \mathbb{Z}_N^*$, and must compute x . Simulator $\mathcal{S}$ simulates the threshold signature game for $\mathcal{A}$ by:

1. Programming the public key to embed the challenge
2. Simulating corrupted party shares as random field elements
3. Answering signing queries using the RSA signing oracle, while handling malicious partial signatures from corrupted parties
4. Extracting an RSA solution from $\mathcal{A}$'s forgery

Let $\mathcal{C} \subseteq [n]$ denote the corruption set selected by $\mathcal{A}$ with $|\mathcal{C}| \leq t - 1$, and let $\mathcal{H} = [n] \setminus \mathcal{C}$ denote honest parties with $|\mathcal{H}| \geq n - t + 1 \geq 1$.

Simulation Construction. **1. Setup Phase:** Simulator $\mathcal{S}$ receives RSA challenge (N, e, y) .

- Publish public key $(N, e, \mathcal{X})$ to $\mathcal{A}$
- Adversary $\mathcal{A}$ selects corruption set $\mathcal{C}$
- For each $i \in \mathcal{C}$: sample $y_i \xleftarrow{\$} \mathbb{Z}_{\phi(N)}$ uniformly at random and provide (x_i, y_i) to $\mathcal{A}$

Note that $\mathcal{S}$ does not know the actual private exponent d or honest parties' shares.

2. Signing Query Simulation: For signing query m_j from $\mathcal{A}$:

- The challenger selects signing coalition $S_j \subseteq [n]$ with $|S_j| = t$
- Obtain reference signature: query RSA signing oracle on m_j to receive $\sigma_j = m_j^d \bmod N$
- **Handle Malicious Partial Signatures:** The adversary $\mathcal{A}$ controls corrupted parties and may cause them to submit arbitrary values as partial signatures
- Simulate honest partial signatures:
 - Expected corrupted contribution (if honest): $\sigma_{\mathcal{C}}^{\text{honest}} \leftarrow \prod_{i \in S_j \cap \mathcal{C}} m_j^{\lambda_i \cdot y_i} \bmod N$ using simulated shares
 - Actual corrupted contribution: $\sigma_{\mathcal{C}}^{\text{actual}} \leftarrow$ the product of actual partial signatures submitted by corrupted parties (potentially malicious)
 - Compute required honest contribution: $\sigma_{\mathcal{H}} \leftarrow \sigma_j \cdot (\sigma_{\mathcal{C}}^{\text{honest}})^{-1} \bmod N$
 - Distribute $\sigma_{\mathcal{H}}$ among honest parties in $S_j \cap \mathcal{H}$:
 - * If $|S_j \cap \mathcal{H}| = 1$: set $\sigma_{i,j} \leftarrow \sigma_{\mathcal{H}}$ for the unique $i \in S_j \cap \mathcal{H}$
 - * If $|S_j \cap \mathcal{H}| \geq 2$: fix $k \in S_j \cap \mathcal{H}$, sample $\sigma_{i,j} \xleftarrow{\$} \mathbb{Z}_N^*$ for $i \in (S_j \cap \mathcal{H}) \setminus \{k\}$, set $\sigma_{k,j} \leftarrow \sigma_{\mathcal{H}} \cdot \left(\prod_{i \in (S_j \cap \mathcal{H}) \setminus \{k\}} \sigma_{i,j} \right)^{-1} \bmod N$
- Provide honest partial signatures $\{\sigma_{i,j}\}_{i \in S_j \cap \mathcal{H}}$ to $\mathcal{A}$; adversary controls corrupted parties' outputs

Indistinguishability Analysis. The simulated view must be indistinguishable from the real protocol execution. Consider:

- **Corrupted Shares Distribution:** In the real protocol, shares $\{y_i\}_{i \in \mathcal{C}}$ are evaluations $y_i = f(x_i) \bmod \phi(N)$ of a random degree- $(t-1)$ polynomial f with $f(0) = d$. By Shamir secret sharing properties, these $|\mathcal{C}| \leq t-1$ evaluations are uniformly distributed in $\mathbb{Z}_{\phi(N)}^{|\mathcal{C}|}$ independent of d . The simulated shares (uniformly random in $\mathbb{Z}_{\phi(N)}$) match this distribution exactly.
- **Partial Signatures Distribution:** For signing coalition S_j :
 - **Corrupted partial signatures:** In both real and simulated executions, malicious parties controlled by $\mathcal{A}$ can output arbitrary values. The simulation allows $\mathcal{A}$ full control over corrupted parties' outputs, matching the real execution.
 - **Honest partial signatures:** In the real protocol, these satisfy $\prod_{i \in S_j \cap \mathcal{H}} \sigma_{i,j} = \sigma_{\mathcal{H}} \pmod{N}$ where $\sigma_{\mathcal{H}}$ is the honest parties' contribution to the complete signature $\sigma_j = m_j^d$. The simulation enforces this same constraint.
 - When $|S_j \cap \mathcal{H}| \geq 2$, individual honest partial signatures are randomized subject only to the product constraint. This matches the information-theoretic view of $\mathcal{A}$: since $\mathcal{A}$ controls fewer than t parties, it cannot learn individual shares of honest parties, only their combined contribution.

Formal Claim: The simulated view is statistically indistinguishable from the real protocol view: for any distinguisher $\mathcal{D}$,

$$|\Pr[\mathcal{D}(\text{RealView}) = 1] - \Pr[\mathcal{D}(\text{SimView}) = 1]| = \text{negl}(\kappa).$$

Forgery Extraction. Suppose $\mathcal{A}$ outputs forgery (m^*, σ^*) with $m^* \notin \mathcal{Q}$ and $(\sigma^*)^e \equiv m^* \pmod{N}$. Then $\sigma^* \equiv (m^*)^d \pmod{N}$ is a valid RSA signature on a message not queried to the oracle. The simulator outputs σ^* as the solution to the RSA problem, successfully breaking the RSA assumption.

Probability Analysis. If $\mathcal{A}$ succeeds with probability ϵ in the EUF-CMA game, and the simulation is statistically indistinguishable (deviation $\text{negl}(\kappa)$), then $\mathcal{S}$ succeeds in solving the RSA problem with probability $\epsilon - \text{negl}(\kappa)$. By contrapositive, if RSA is secure (no PPT adversary succeeds with non-negligible probability), then ϵ must be negligible.

This completes the reduction, establishing EUF-CMA security:

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}(\kappa, t, n) \leq \text{Adv}_{\mathcal{B}}^{\text{RSA}}(\kappa) + \text{negl}(\kappa).$$

6 Implementation and Evaluation

This section validates two key practical claims: (1) that large coordinate values do not create computational bottlenecks, and (2) that the algorithm's simplicity enables deployment on resource-constrained hardware security modules.

6.1 Performance Evaluation and Comparison

We validate that large coordinate values do not create performance bottlenecks through a Python prototype implementation using `gmpy2` [1] on an Apple M1 Max processor with 3072-bit RSA and $e = 3$. Table 3 compares standard RSA against threshold configurations (3, 5), (5, 9), and (9, 9) over 100 iterations using coordinate set (2, 9).

Table 3. Performance comparison: Standard RSA vs. Threshold RSA with integer coordinates (3072-bit modulus, $e = 3$, 100 iterations, Apple M1 Max)

Configuration	Avg. Total Time	Per-Party Time	Per-Party Overhead
Standard RSA	6.18 ms	6.18 ms	N/A
Threshold (3-of-5)	19.56 ms	6.50 ms	5%
Threshold (5-of-9)	33.86 ms	6.75 ms	9%
Threshold (9-of-9)	66.96 ms	7.45 ms	20%

Key Finding. The time for combining the partial signatures is negligible compared to the per-party signing time. Per-party signing time remains 6.5–7.5 ms across all configurations, representing 5–20% overhead compared to standard RSA (6.2 ms). This overhead is acceptable and not a deployment blocker—each party performs a single modular exponentiation followed by integer arithmetic for Lagrange coefficients. Even for (9, 9) configurations using 32-digit coordinate values (Table 2), coefficient computation remains dominated by the core exponentiation operation. Total signing time scales linearly with threshold t due to sequential execution in the test harness; in parallel deployments where parties compute simultaneously, effective latency is ≈ 6.5 –7.5 ms.

6.2 Validation: Implementability on Resource-Constrained Hardware

The critical validation is demonstrating that the integer coordinate framework *can be implemented* on severely resource-constrained hardware. Many existing threshold RSA schemes—polynomial arithmetic frameworks [10], interactive multi-round protocols [2], zero-knowledge proof systems—are impractical for HSMs, secure enclaves, and hardware wallets. If the framework cannot execute within such constraints, it cannot serve its intended purpose of enabling threshold RSA for Intel SGX code signing.

Ledger Implementation. The threshold signing infrastructure is implemented on Ledger Stax devices to generate valid RSA- $e = 3$ signatures for SGX enclave code signing. Each Ledger holds one secret share and runs Algorithm 2 (lines 2–7) within its secure element to produce a partial signature with exponent $e = 3$. These partial signatures are combined off-device into a single RSA signature

that satisfies SGX’s hard-coded $e = 3$ check. Ledger hardware wallets represent an extreme environment: approximately 4KB RAM, no floating-point unit, limited flash storage, and restricted C development. The implementation uses hierarchical key management where each device derives an RSA key pair from its hardware-backed seed (via BIP-39/BIP-32) as a key-wrapping key (KWK) for protecting the threshold share y_i . The signing protocol executes entirely within the secure element: decrypt share y_i using the KWK, retrieve pre-computed Lagrange coefficient λ_i (using the $(2, 8)$ -integer coordinate set with all values fitting in 32-bit integers), and generate partial signature $\sigma_i = m^{\lambda_i \cdot y_i} \bmod N$ through modular exponentiation. The entire signing logic occupies approximately 1,000 lines of C code using the Ledger SDK’s `cx_math` library, demonstrating that the framework is *fundamentally compatible* with commodity secure hardware constraints—making it viable for Intel SGX enclaves, HSMs, and similar environments requiring threshold RSA with $e = 3$. End-to-end latency (host-to-Ledger communication plus on-device signing) is approximately 1.3 seconds per device, which is acceptable for hardware wallet deployments where user approval is required before each signature operation.

7 Conclusion

This work establishes the integer coordinate framework for threshold RSA signatures, addressing the fundamental tension between Shamir secret sharing reconstruction (which requires Lagrange coefficient computation modulo $\phi(N)$) and RSA security (which requires $\phi(N)$ to remain secret). The approach is based on the mathematical observation that appropriately selected interpolation coordinates yield integer-valued Lagrange coefficients, eliminating all modular arithmetic modulo $\phi(N)$ from the online signing protocol.

7.1 Fundamental Open Problem

The central limitation is the absence of efficient constructive algorithms for Problem 1. All coordinate families presented result from heuristic computational search. The following questions remain open:

1. **Universal Existence:** Do (t, n) -integer families exist for all (t, n) ? If not, characterize parameter regimes admitting solutions.
2. **Constructive Algorithms:** Develop polynomial-time algorithms (in n , t , and output size) for coordinate generation, or establish complexity-theoretic barriers.
3. **Size Bounds:** Characterize the asymptotic growth of $\min_{\mathcal{X}} \max_i |x_i|$ as a function of (t, n) . Current empirical data (Table 2) shows exponential growth from $n = 8$ to $n = 9$; determine whether this is fundamental or an artifact of search heuristics.
4. **Algebraic Structure:** Connect integer coordinate families to established mathematical frameworks (combinatorial designs, lattice theory, Diophantine equations) to enable theoretical characterization.

Resolution of these questions will determine whether the integer coordinate framework scales to large n (with efficiently computable small coordinates) or is inherently limited to small-to-moderate group sizes.

7.2 Concluding Remarks

The integer coordinate framework provides the most efficient known approach to threshold RSA with $e = 3$ support and proven unforgeability for small-to-moderate group sizes. Among existing $e = 3$ -compatible schemes, the integer coordinate method uniquely achieves the combination of: (1) EUF-CMA security (unforgeability) against malicious adversaries under the standard RSA assumption alone (Theorem 1), (2) standard $O(\kappa)$ -bit share sizes, and (3) perfect secrecy preservation of Shamir secret sharing. For deployments requiring robustness against denial-of-service, an optional DLEQ-based verification extension (Appendix A) provides practical security with approximately $2\times$ overhead, though formal analysis under adaptive adversaries remains future work.

The integer coordinate framework demonstrates that fundamental cryptographic challenges—here, the tension between secret sharing reconstruction and RSA security—can be resolved through careful mathematical structure rather than additional cryptographic assumptions. By eliminating modular arithmetic modulo $\phi(N)$ via integer-valued Lagrange coefficients, the approach achieves proven unforgeability, perfect secrecy, and practical efficiency simultaneously. The successful Ledger deployment validates that this theoretical elegance translates to real-world feasibility in severely constrained environments.

For the practically important regime of small-to-moderate group sizes ($n \leq 9$), the integer coordinate framework provides immediate, deployment-ready threshold RSA with $e = 3$ support—addressing a long-standing limitation in cryptographic infrastructure ranging from Intel SGX enclaves to hardware security modules. Resolution of the coordinate construction problem (Section 7.2) represents the path to broader applicability, though current results already serve a significant deployment space. The clean separation between unforgeability (proven) and robustness (practical extension) reflects mature cryptographic engineering: providing strong theoretical foundations while acknowledging practical requirements and open problems honestly.

References

1. Gmpy2: Multiple-precision arithmetic for python. Python Package Index, 2021.
2. Dario Catalano, Rosario Gennaro, and Shai Halevi. Computing inverses over a shared secret modulus. In Bart Preneel, editor, *Advances in Cryptology — EUROCRYPT 2000*, pages 190–206, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
3. David Chaum and Torben Pryds Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *Advances in Cryptology — CRYPTO’ 92*, pages 89–105, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.
4. Intel Corporation. Intel software guard extensions programming reference. Intel Developer Documentation, 2016.

5. Ronald Cramer, Ivan Bjerre Damgrd, and Jesper Buus Nielsen. *Secure Multiparty Computation and Secret Sharing*. Cambridge University Press, USA, 1st edition, 2015.
6. Ivan Damgård and Kasper Dupont. Efficient threshold rsa signatures with general moduli and no extra assumptions. In Serge Vaudenay, editor, *Public Key Cryptography - PKC 2005*, pages 346–361, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
7. Ivan B. Damgård and Maciej Koprowski. Practical threshold rsa signatures without a trusted dealer. *BRICS Report Series*, 7(30), Jan. 2000.
8. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In *Advances in Cryptology—CRYPTO’89 Proceedings*, pages 307–315. Springer, 1989.
9. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
10. Yair Frankel and Yvo Desmedt. Parallel reliable threshold multisignature. *Dept. of Elect. Eng. and Computer Sci., Univ. of Wisconsin-Milwaukee, Tech. Rep. TR-92-04-02*, 1992.
11. Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Robust threshold dss signatures. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 354–371. Springer, 1996.
12. Shafi Goldwasser, Silvio Micali, and Ronald L Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on computing*, 17(2):281–308, 1988.
13. Amir Herzberg, Stanislaw Jarecki, Hugo Krawczyk, and Moti Yung. Proactive secret sharing or: How to cope with perpetual leakage. pages 339–352, 1995.
14. Marc Joye and Yan Michalevsky. Rsa signatures under hardware restrictions. In *Proceedings of the 2018 Workshop on Attacks and Solutions in Hardware Security, ASHES ’18*, page 51–54, New York, NY, USA, 2018. Association for Computing Machinery.
15. Kathleen Moriarty, Burt Kaliski, Jakob Jonsson, and Andreas Rusch. Pkcs #1: Rsa cryptography specifications version 2.2. RFC 8017, November 2016.
16. Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.
17. R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, feb 1978.
18. Ron Rivest and Robert Silverman. Are ‘strong’ primes needed for RSA. Cryptology ePrint Archive, Paper 2001/007, 2001.
19. Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, nov 1979.
20. Victor Shoup. Practical threshold signatures. In Bart Preneel, editor, *Advances in Cryptology — EUROCRYPT 2000*, pages 207–220, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
21. Lawrence C Washington. *Introduction to cyclotomic fields*, volume 83. Springer Science & Business Media, 1997.
22. Qiu-Liang Xu and Tzer-Shyong Chen. An efficient threshold rsa digital signature scheme. *Applied mathematics and computation*, 166(1):25–34, 2005.

A Optional DLEQ-Based Robustness Extension

This appendix presents an optional extension to the threshold RSA scheme that provides robustness against denial-of-service attacks through verification of par-

tial signatures. The extension employs Discrete Logarithm Equivalence (DLEQ) proofs to enable parties to detect and identify malicious behavior during signature generation.

Optional DLEQ-Based Robustness Extension. For practical deployments requiring robustness against denial-of-service, we develop non-interactive partial signature verification using DLEQ proofs with computational overhead of approximately $2\times$ compared to the base scheme—substantially lower than traditional robust threshold schemes. This extension is presented as an optional enhancement; the core unforgeability result (Theorem 1) stands independently.

Relationship to Main Result. The core contribution of this work is the unforgeability result (Theorem 1) for the base integer coordinate scheme, which requires no additional cryptographic assumptions beyond RSA. The DLEQ-based verification mechanism presented in this appendix is an *optional enhancement* for deployments requiring robustness properties. While this extension has been successfully deployed in practice (Section 6) and follows standard DLEQ proof techniques, formal security analysis under adaptive malicious adversaries requires careful treatment of witness dependencies. The simulation challenge arises because witness values $w_i = m_0^{y_i}$ commit to shares before the adversary’s strategy is known, creating extraction problems in security proofs. We present the mechanism with an honest assessment of its practical value and theoretical limitations.

Protocol Approach. The verification mechanism employs witness-based Non-Interactive Discrete Logarithm Equivalence (DLEQ) proofs [3,9], which allow each party to prove that their partial signature was correctly computed from their share without revealing the share itself.

A.1 Protocol Specification

This section provides the complete algorithmic specification of the DLEQ-based verification protocol, including witness generation, proof construction, verification algorithms, and the modified signing protocol.

Protocol Overview The witness-based DLEQ approach enables verification of partial signatures by proving that the same secret share was used in both a public witness value and the partial signature for a given message. The protocol consists of three phases:

1. **Witness Generation:** Each party publishes a commitment to their share
2. **Proof Generation:** During signing, each party generates a DLEQ proof alongside their partial signature
3. **Verification:** Any party can verify the proof to ensure correctness of the partial signature

A.2 Witness Generation Protocol

After the trusted dealer distributes shares in Algorithm 1, each party generates a witness value.

Algorithm 3 WitnessGen(i, y_i, m_0, N) – Witness Value Generation

- 1: **Input:** Party index i , share $y_i \in \mathbb{Z}_{\phi(N)}$, witness message $m_0 \in \mathbb{Z}_N^*$, modulus N
 - 2: **Witness Message Selection:** If not already specified, set $m_0 \leftarrow H(\text{"threshold-rsa-witness"}\|N)$ where H is a cryptographic hash function mapping to $\mathbb{Z}_N^*$
 - 3: **Witness Computation:** Compute $w_i \leftarrow m_0^{y_i} \bmod N$
 - 4: **Publication:** Broadcast w_i to all parties via authenticated channel
 - 5: **Output:** Witness value $w_i \in \mathbb{Z}_N$
-

Security Note. The witness value $w_i = m_0^{y_i} \bmod N$ does not reveal the share y_i under the RSA assumption (computing discrete logarithms modulo N without knowing the factorization is equivalent to breaking RSA). However, it serves as a public commitment that can be used to verify future partial signatures.

A.3 DLEQ Proof Construction

The DLEQ proof, Algorithm 4, demonstrates that $\log_{m_0}(w_i^{\lambda_i}) = \log_m(\sigma_i)$, i.e., that both values have the same discrete logarithm (namely, $\lambda_i \cdot y_i$) with respect to different bases.

Non-Interactive Conversion. The Fiat-Shamir heuristic [9] converts the interactive Chaum-Pedersen protocol into a non-interactive proof by deriving the challenge c from a hash of the protocol transcript. This eliminates interaction at the cost of requiring the random oracle model.

A.4 DLEQ Proof Verification

Refer to Algorithm 5 for the DLEQ proof verification algorithm.

A.5 Modified Signing Protocol with DLEQ

The threshold signing protocol Algorithm 2 is modified to Algorithm 6 to include DLEQ proof generation and verification:

Algorithm 4 DLEQProve($y_i, \lambda_i, m_0, w_i, m, \sigma_i, N$) – DLEQ Proof Generation

- 1: **Input:** Share y_i , Lagrange coefficient λ_i , witness message m_0 , witness w_i , message m , partial signature σ_i , modulus N
 - 2: **Common Parameters:** Bases $(g_1, h_1) = (m_0, w_i^{\lambda_i})$ and $(g_2, h_2) = (m, \sigma_i)$
 - 3: **Shared Secret:** $s = \lambda_i \cdot y_i$ (such that $h_1 \equiv g_1^s \pmod{N}$ and $h_2 \equiv g_2^s \pmod{N}$)
 - 4:
 - 5: **Commitment Phase:**
 - 6: Sample random $r \xleftarrow{\$} \mathbb{Z}_{\phi(N)}$
 - 7: Compute commitments:
 - 8: $a_1 \leftarrow m_0^r \pmod{N}$
 - 9: $a_2 \leftarrow m^r \pmod{N}$
 - 10:
 - 11: **Challenge Phase (Fiat-Shamir):**
 - 12: Compute challenge: $c \leftarrow H(m_0 \| w_i^{\lambda_i} \| m \| \sigma_i \| a_1 \| a_2 \| N)$
 - 13: where H is a cryptographic hash function mapping to $\mathbb{Z}_{2^\kappa}$ (for security parameter κ)
 - 14:
 - 15: **Response Phase:**
 - 16: Compute response: $z \leftarrow r + c \cdot s$ (computed over $\mathbb{Z}$, no modular reduction)
 - 17:
 - 18: **Output:** Proof $\pi = (c, z)$
-

Algorithm 5 DLEQVerify($\pi, m_0, w_i, \lambda_i, m, \sigma_i, N$) – DLEQ Proof Verification

- 1: **Input:** Proof $\pi = (c, z)$, witness message m_0 , witness w_i , Lagrange coefficient λ_i , message m , partial signature σ_i , modulus N
 - 2: **Common Parameters:** Bases $(g_1, h_1) = (m_0, w_i^{\lambda_i})$ and $(g_2, h_2) = (m, \sigma_i)$
 - 3:
 - 4: **Recompute Commitments:**
 - 5: $a'_1 \leftarrow m_0^z \cdot (w_i^{\lambda_i})^{-c} \pmod{N}$
 - 6: $a'_2 \leftarrow m^z \cdot \sigma_i^{-c} \pmod{N}$
 - 7:
 - 8: **Recompute Challenge:**
 - 9: $c' \leftarrow H(m_0 \| w_i^{\lambda_i} \| m \| \sigma_i \| a'_1 \| a'_2 \| N)$
 - 10:
 - 11: **Verification Check:**
 - 12: **if** $c' = c$ **then**
 - 13: **Output:** 1 (accept)
 - 14: **else**
 - 15: **Output:** 0 (reject)
 - 16: **end if**
-

Algorithm 6 ThresholdSignDLEQ($m, S, \{(x_i, y_i, w_i)\}_{i \in S}, N, e, \mathcal{X}, m_0$) – Signing with DLEQ Verification

- 1: **Input:** Message m , coalition S with $|S| = t$, shares and witnesses $\{(x_i, y_i, w_i)\}_{i \in S}$, public key $(N, e, \mathcal{X})$, witness message m_0
 - 2:
 - 3: **Partial Signature Generation (each party P_i for $i \in S$):**
 - 4: Compute $\lambda_i \leftarrow \prod_{j \in S \setminus \{i\}} \frac{x_j}{x_j - x_i}$
 - 5: Compute $\sigma_i \leftarrow m^{\lambda_i \cdot y_i} \bmod N$ (handling negative λ_i as in Algorithm 2)
 - 6: Generate DLEQ proof: $\pi_i \leftarrow \text{DLEQProve}(y_i, \lambda_i, m_0, w_i, m, \sigma_i, N)$
 - 7: Broadcast (σ_i, π_i)
 - 8:
 - 9: **Verification and Combination (combiner or any party):**
 - 10: Initialize valid set $S' \leftarrow \emptyset$
 - 11: **for** each $i \in S$ **do**
 - 12: Compute $\lambda_i \leftarrow \prod_{j \in S \setminus \{i\}} \frac{x_j}{x_j - x_i}$
 - 13: **if** DLEQVerify($\pi_i, m_0, w_i, \lambda_i, m, \sigma_i, N$) = 1 **then**
 - 14: $S' \leftarrow S' \cup \{i\}$
 - 15: **else**
 - 16: Report: “Party i provided invalid DLEQ proof”
 - 17: **end if**
 - 18: **end for**
 - 19: **if** $|S'| < t$ **then**
 - 20: **abort** with “Insufficient valid partial signatures”
 - 21: **end if**
 - 22: Compute complete signature: $\sigma \leftarrow \prod_{i \in S'} \sigma_i \bmod N$
 - 23: **Output:** σ
-